NEXORA – SMART CITY CITIZEN SERVICE REQUEST MANAGEMENT SYSTEM

Project Title: NEXORA – Smart City Citizen Service Request Management System

Technology Used: HTML, CSS, PHP, MySQL

Hosting: InfinityFree

Hosted URL: https://nexora-city.infinityfreeapp.com/

1. Selected Problem Statement

Problem Statement

In a smart city, citizens need a simple way to report public service issues such as street-light faults, sanitation problems, road issues and other municipal service requirements. Manual complaint handling can make it difficult to track requests, assign responsibility and check request status.

To solve this problem, we developed NEXORA, a web-based Smart City Citizen Service Request Management System. Citizens can submit service requests with their area, service type, description, priority and assigned department details. The system stores the information in a MySQL relational database and allows records to be viewed and searched.

The application uses PHP for backend processing and MySQL for storing and retrieving data. The completed application is hosted online using InfinityFree.

2. Objective

The main objective of the project is to develop a simple, user-friendly and secure database-driven citizen service request website.

- To create a meaningful online platform for submitting citizen service requests.
- To store citizen, area and service request information in a MySQL database.
- To provide a labelled HTML form with client-side validation.
- To use PHP for server-side validation and database processing.
- To demonstrate INSERT and SELECT operations.
- To provide search/filter functionality for service requests.
- To use prepared statements with ? placeholders and parameter binding.
- To display database values safely using htmlspecialchars().
- To deploy the website online using InfinityFree.

3. Main Features

1. Citizen Service Request Form

Citizens can enter request details such as citizen name, area, service type, description, priority and assigned department.

2. Client-Side Validation

Required HTML form fields are validated before submission.

3. Secure Request Submission

PHP validates the submitted data and stores valid requests in MySQL using prepared statements.

4. Request Confirmation

After successful insertion, the application displays the generated Request ID and request status.

5. Record List

Stored requests can be displayed in the browser using SELECT queries.

6. Search / Filter

Users can search and filter request records according to available criteria.

7. Secure Input Handling

Prepared statements and parameter binding are used instead of directly joining user input into SQL.

8. Safe Output

htmlspecialchars() is used while displaying data to reduce HTML/script injection risk.

9. Hosted Website

The application is deployed on InfinityFree and works through a public URL without login.

4. Technology Stack

Technology	Purpose
HTML	Creates the structure of web pages and forms
CSS	Provides styling, layout and visual presentation
PHP	Handles backend logic, validation and database processing
MySQL	Stores citizens, areas and service request data
phpMyAdmin	Used to manage/import the hosted MySQL database
XAMPP	Used for local PHP and MySQL development
InfinityFree	Used to host the website and MySQL database
VS Code	Used to write and edit project files

5. Database Design

The project uses a MySQL relational database. The main tables are areas, citizens and service_requests.

Areas Table

Column	Type	Key / Constraint
area_id	INT	Primary Key, Auto Increment
area_name	VARCHAR	NOT NULL

Purpose

The areas table stores service areas used by citizen service requests.

Citizens Table

Column	Type	Key / Constraint
citizen_id	INT	Primary Key, Auto Increment
citizen_name	VARCHAR	NOT NULL

Purpose

The citizens table stores citizen information referenced by service requests.

Service Requests Table

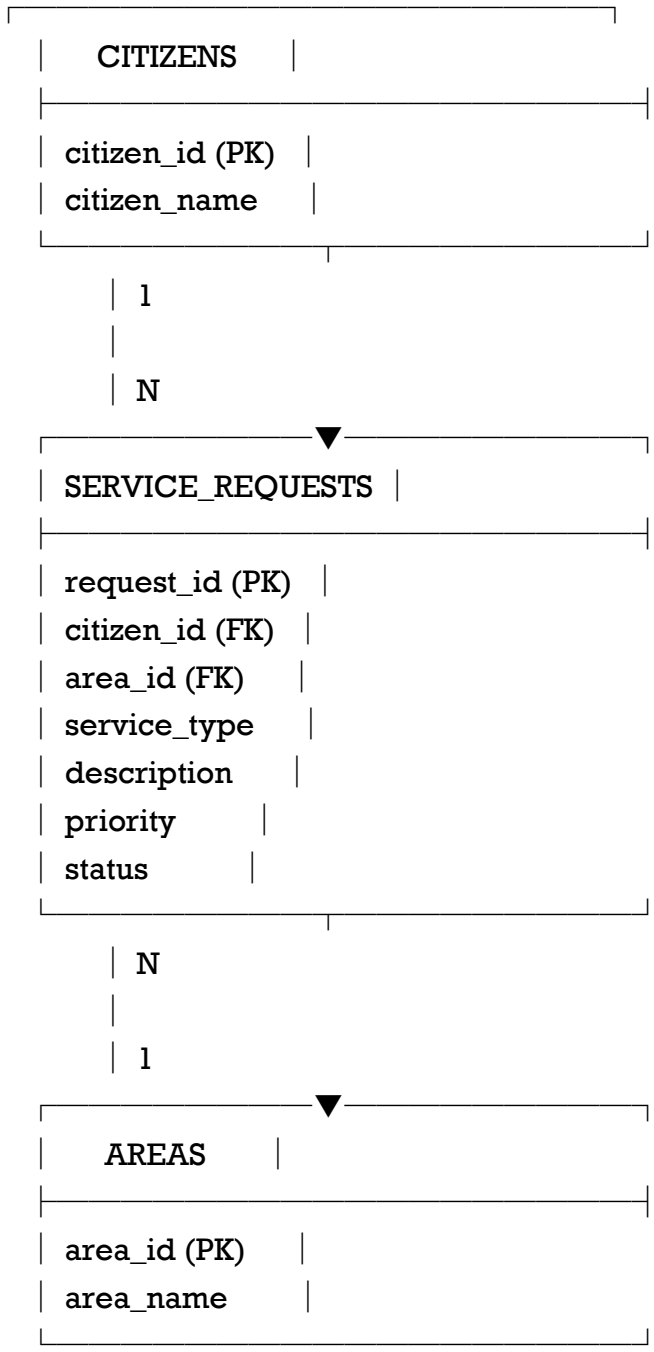
Column	Type	Key / Constraint
request_id	INT	Primary Key, Auto Increment
citizen_id	INT	Foreign Key, NOT NULL
area_id	INT	Foreign Key, NOT NULL
service_type	VARCHAR	NOT NULL
description	TEXT	NOT NULL
priority	VARCHAR	NOT NULL
request_date	DATE/DATETIME	NOT NULL
assigned_department	VARCHAR	NOT NULL
status	VARCHAR	NOT NULL
resolved_date	DATE/DATETIME	Nullable

Purpose

The service_requests table stores citizen service requests, priority, department, status and resolution information.

6. Simple ER Diagram

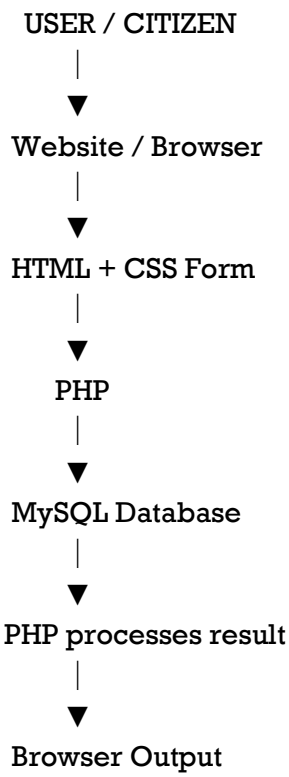
The relationship between the main tables is shown below:



Relationship Explanation

One citizen can create multiple service requests. One area can be associated with multiple service requests. Foreign keys citizen_id and area_id connect the service_requests table with citizens and areas.

7. Application Flow



Detailed Flow

Step 1 — User opens website

The citizen opens the NEXORA website through the hosted URL.

Step 2 — User enters request details

The citizen fills the labelled HTML form with required request information.

Step 3 — Client-side validation

The browser checks required form fields before submission.

Step 4 — PHP processing

PHP receives submitted data and performs server-side validation.

Step 5 — Database connection

PHP connects to MySQL using the project configuration files.

Step 6 — SQL operation

PHP performs INSERT, SELECT or search/filter operations using prepared statements where user input is involved.

Step 7 — Database response

MySQL stores or retrieves the required records.

Step 8 — Browser output

PHP generates the response page and safely displays output using htmlspecialchars().

In short: Browser → HTML/CSS → PHP → MySQL → PHP → Browser

8. SQL Operations Used

1. INSERT

INSERT is used to add a new citizen service request to the database.

INSERT INTO service_requests (...) VALUES (?, ?, ?, ?, ...);

2. SELECT

SELECT is used to retrieve stored service request records and display them in the browser.

SELECT * FROM service_requests;

3. SEARCH / FILTER

Search/filter uses SELECT with conditions and safely bound user values.

SELECT ... WHERE field LIKE ?;

9. Prepared Statements

Prepared statements are used to make database operations safer. Instead of directly joining user input with an SQL query, the SQL statement uses ? placeholders and the actual values are supplied separately using parameter binding.

Example

SELECT * FROM service_requests WHERE description LIKE ?;

The ? represents a value supplied separately through parameter binding.

Why Prepared Statements Are Used

- They help reduce SQL Injection risk.
- They separate SQL commands from user input.
- They handle special characters such as apostrophes safely.
- They improve the security and reliability of search and insert operations.

Security Test

The search value O'Reilly was tested on the hosted website. The result showed 1 matching record with no SQL error, demonstrating safe handling of the apostrophe in the search input.

10. Server-Side Validation

Client-side validation can improve user experience, but it can be bypassed. Therefore, PHP validates submitted data again on the server before executing the database operation. Required fields are checked and invalid input is not directly trusted.

11. htmlspecialchars()

When values are displayed in HTML, htmlspecialchars() is used to encode special characters. This helps prevent stored data from being interpreted directly as HTML or script content in the browser.

12. Hosted Application URL

Live Website

https://nexora-city.infinityfreeapp.com/

The project has been deployed using InfinityFree. The hosted application works without requiring users to run XAMPP locally. Database credentials are private and are not shown in this report or screenshots.

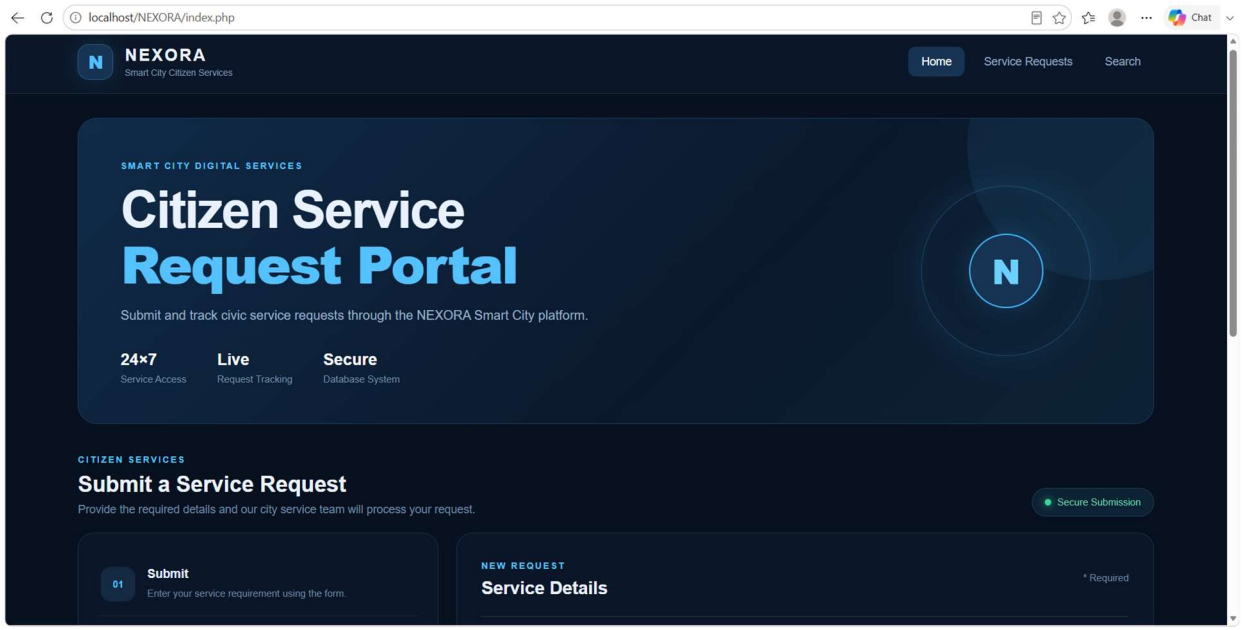
13. Test Cases and Outputs

Test Case	Test Performed	Expected / Observed Output	Status
TC01	Open website	Home/data-entry page opens	PASS
TC02	Submit live form	Request inserted successfully; Request ID #6	PASS
TC03	View requests	6 total requests displayed	PASS
TC04	Check Pending count	Pending: 4	PASS
TC05	Check Resolved count	Resolved: 1	PASS
TC06	Check High Priority count	High Priority: 5	PASS
TC07	Search O'Reilly	1 matching record; no SQL error	PASS
TC08	Open hosted URL	Website accessible without login	PASS

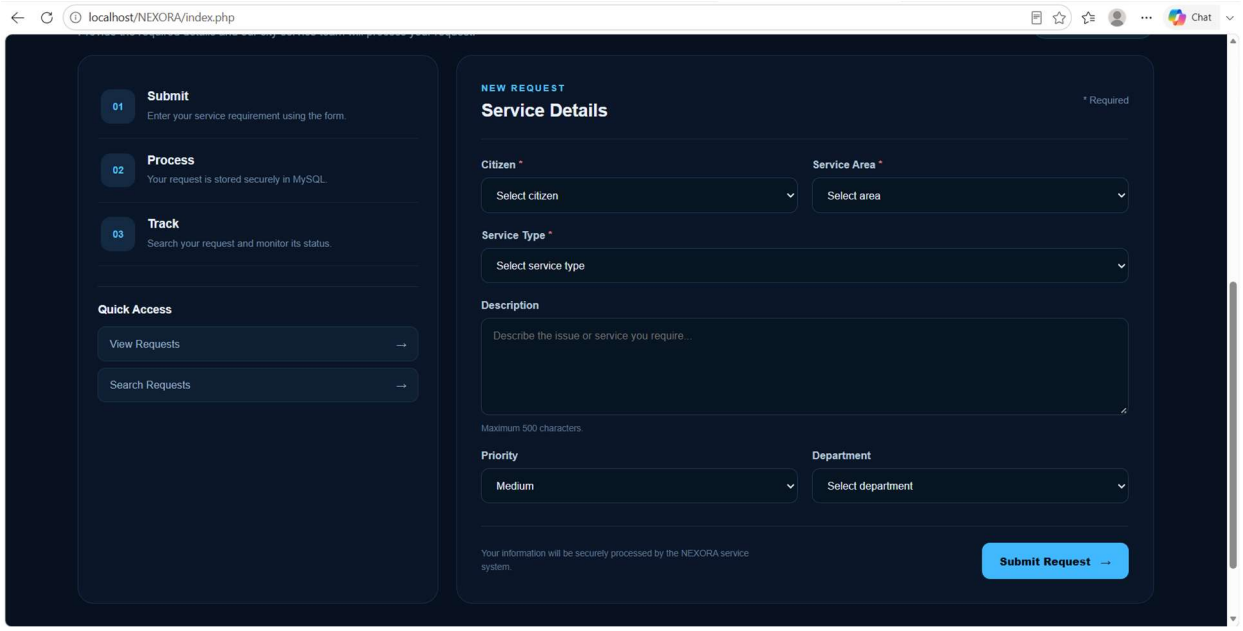
14. Output Screenshots

Screenshot 1 — Home / Data-Entry Form

Local XAMPP home page and NEXORA landing section.

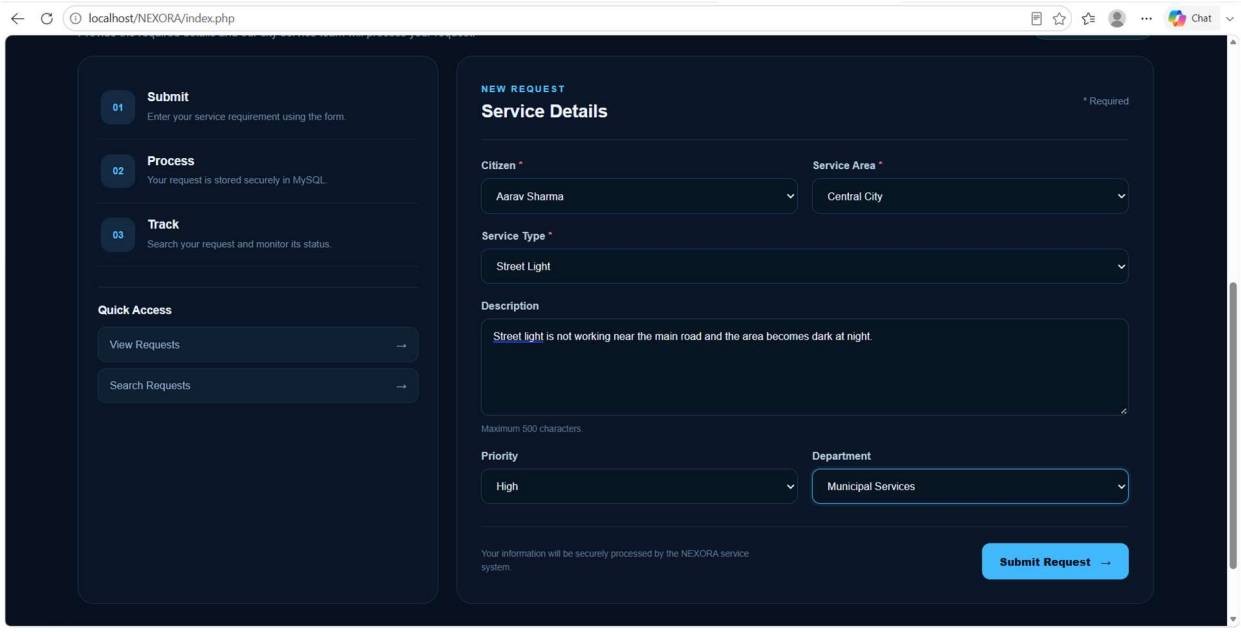


Local XAMPP service-request data-entry form.



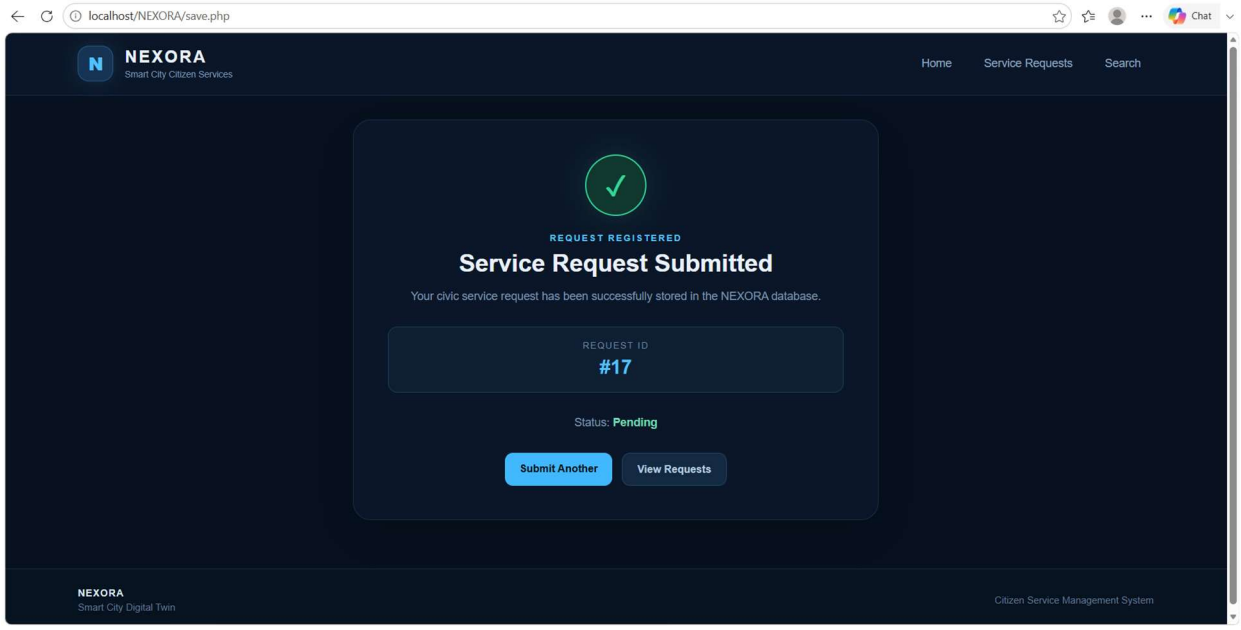
Screenshot 2 — Completed Form Before Submission

Completed service-request form with citizen, area, service, description, priority and department selected.



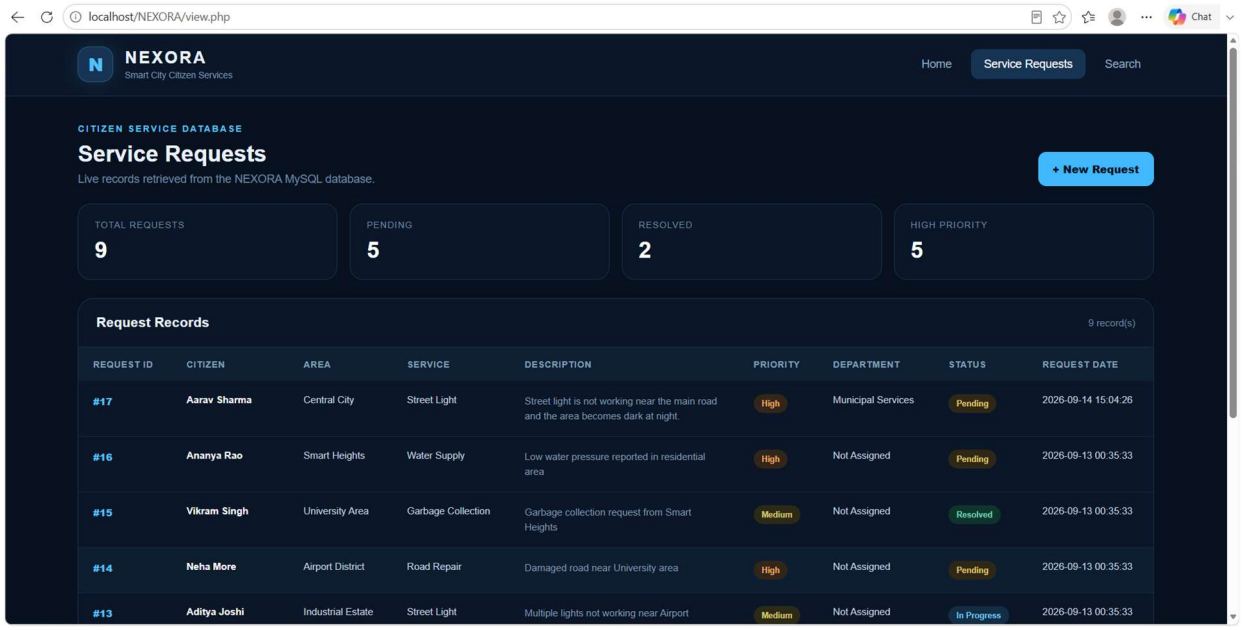
Screenshot 3 — Successful Insert Confirmation

Successful INSERT confirmation showing the generated request ID and Pending status.

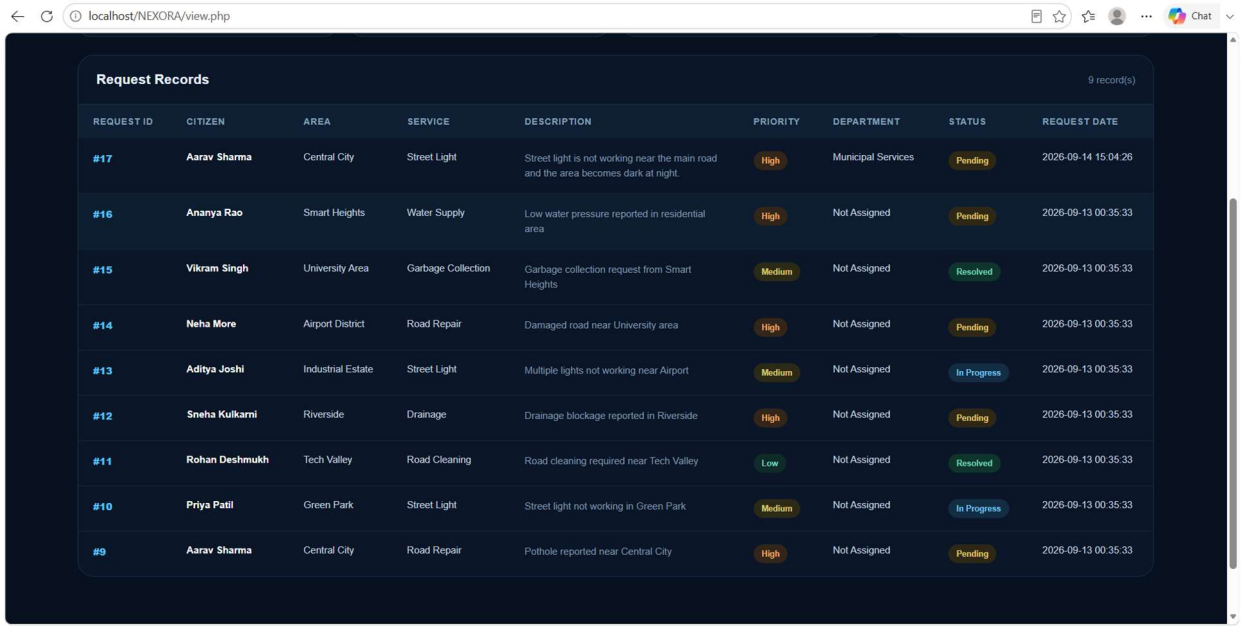


Screenshot 4 — Record List Showing Saved Data

Local record-list page showing request statistics and saved service requests.

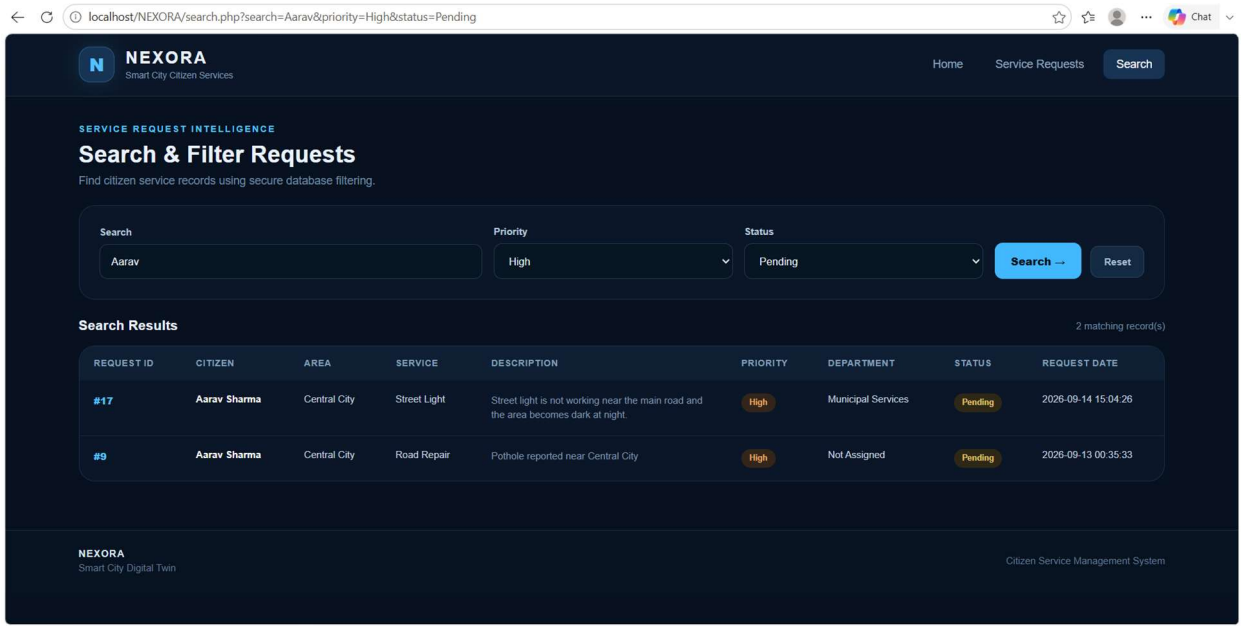


Lower portion of the local record list showing multiple stored requests.

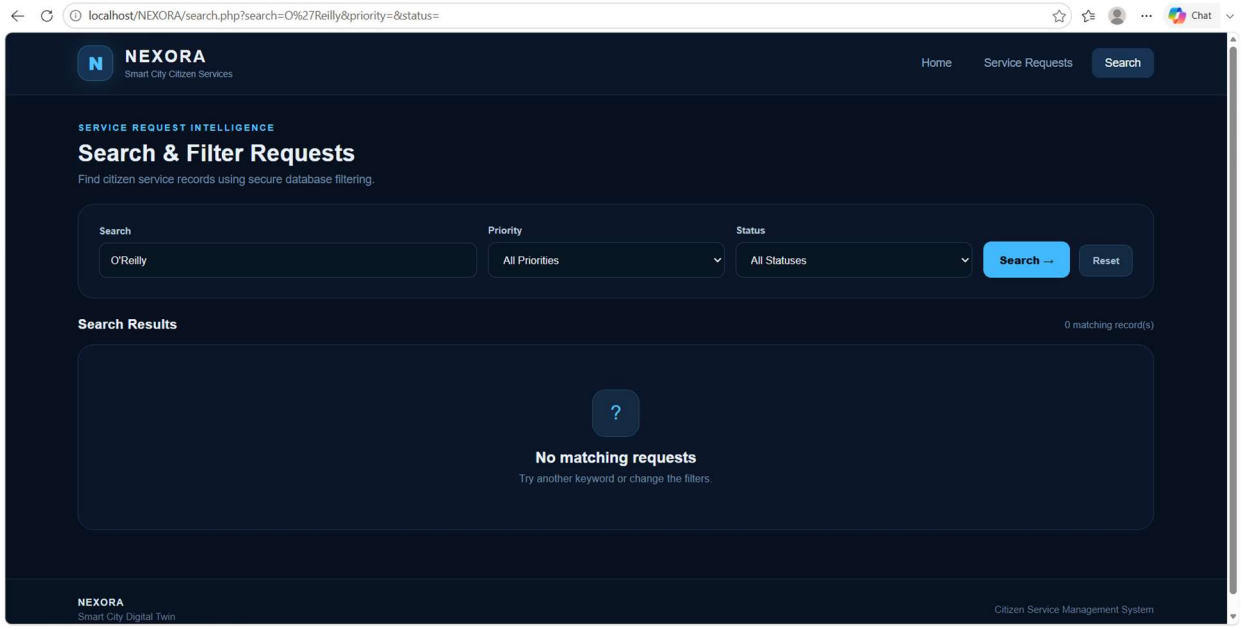


Screenshot 5 — Search / Filter Working

Local search/filter test using Aarav with High priority and Pending status.

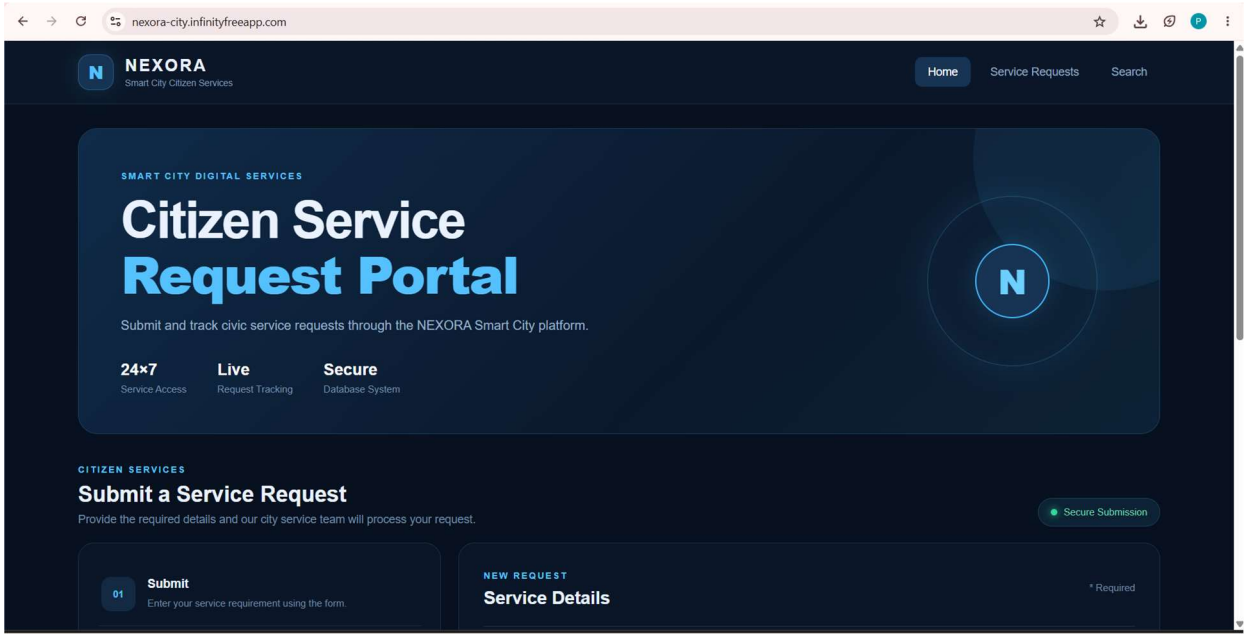


Local O'Reilly input test demonstrating safe handling of a quote character without a SQL error.

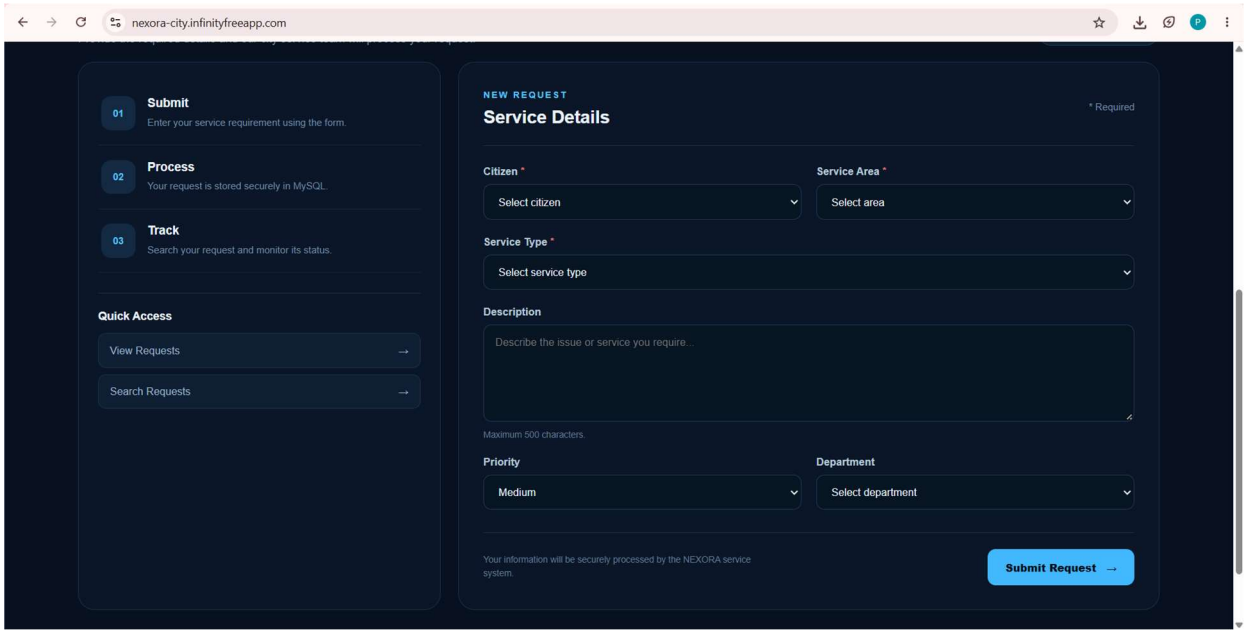


Screenshot 6 — Hosted Application Verification

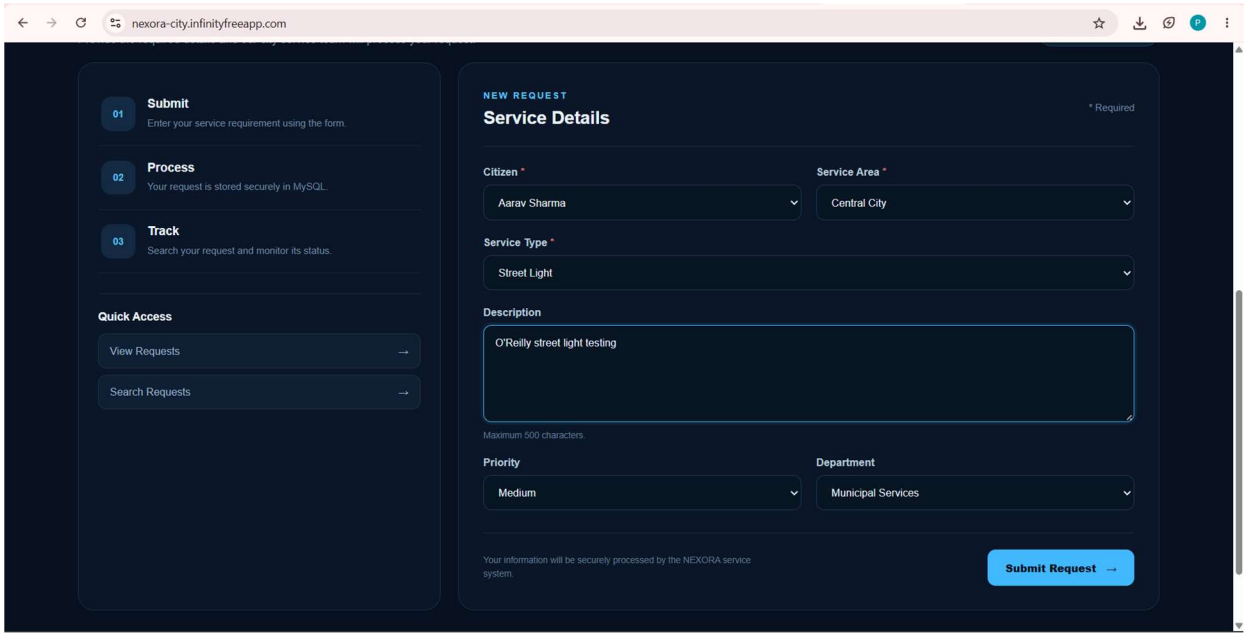
Hosted NEXORA home page showing the deployed portal.



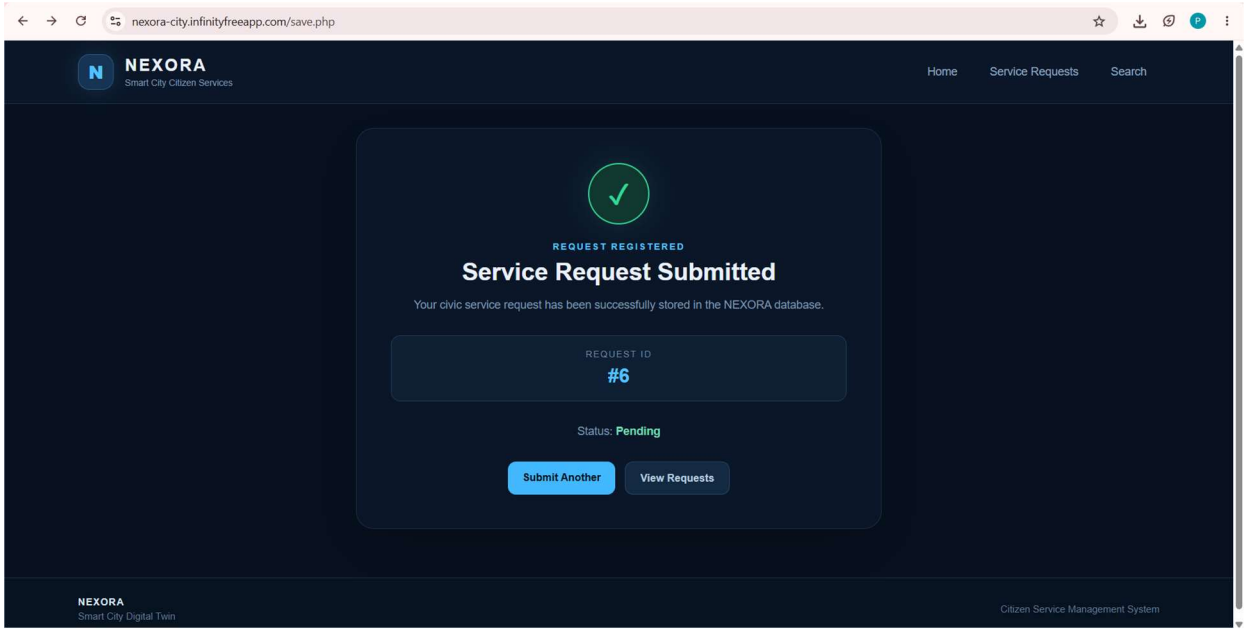
Hosted service-request form in its initial state.



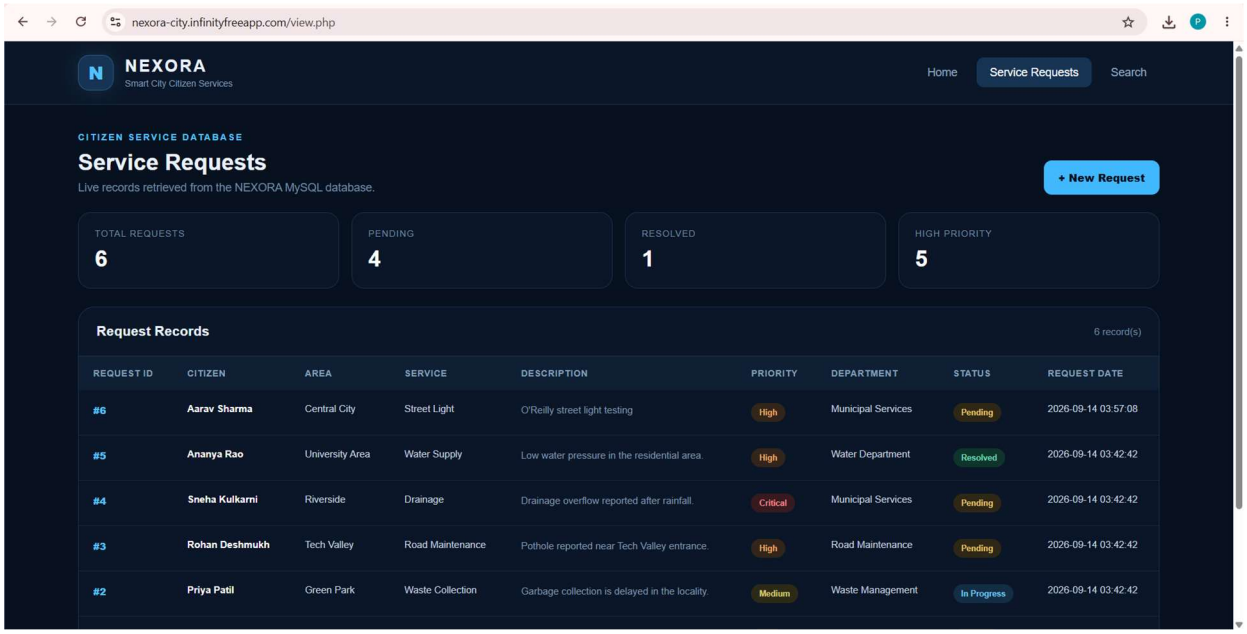
Hosted service-request form filled with O'Reilly security-test data.



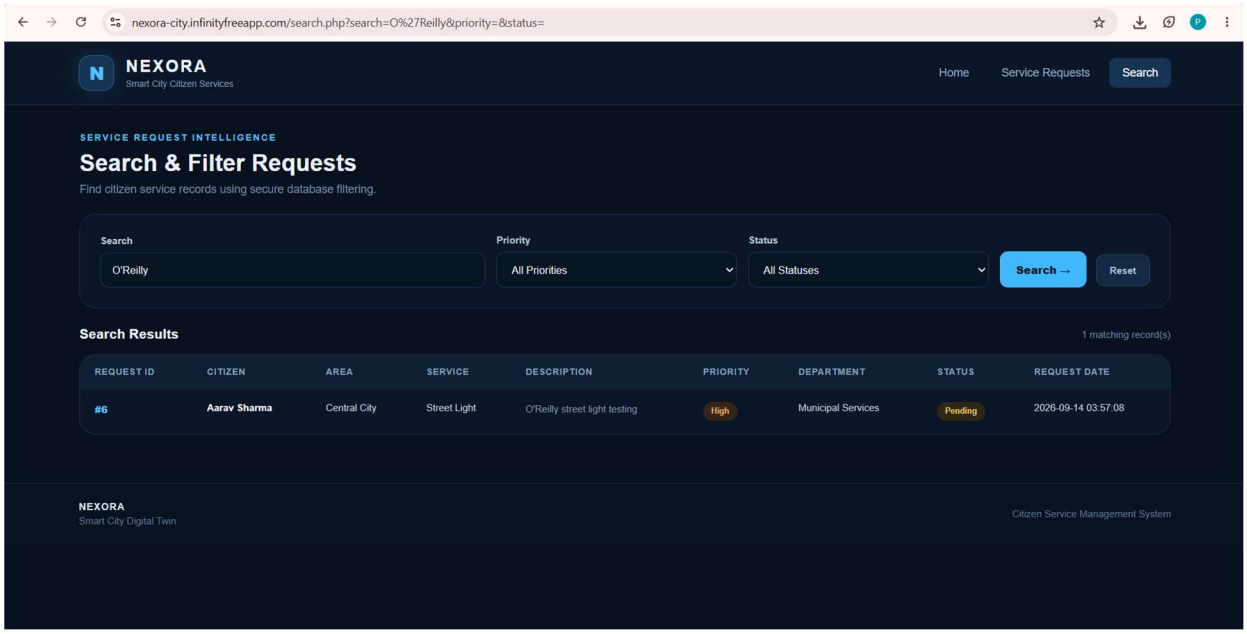
Hosted InfinityFree successful insert confirmation for request.



Hosted InfinityFree service-request list with live database records.

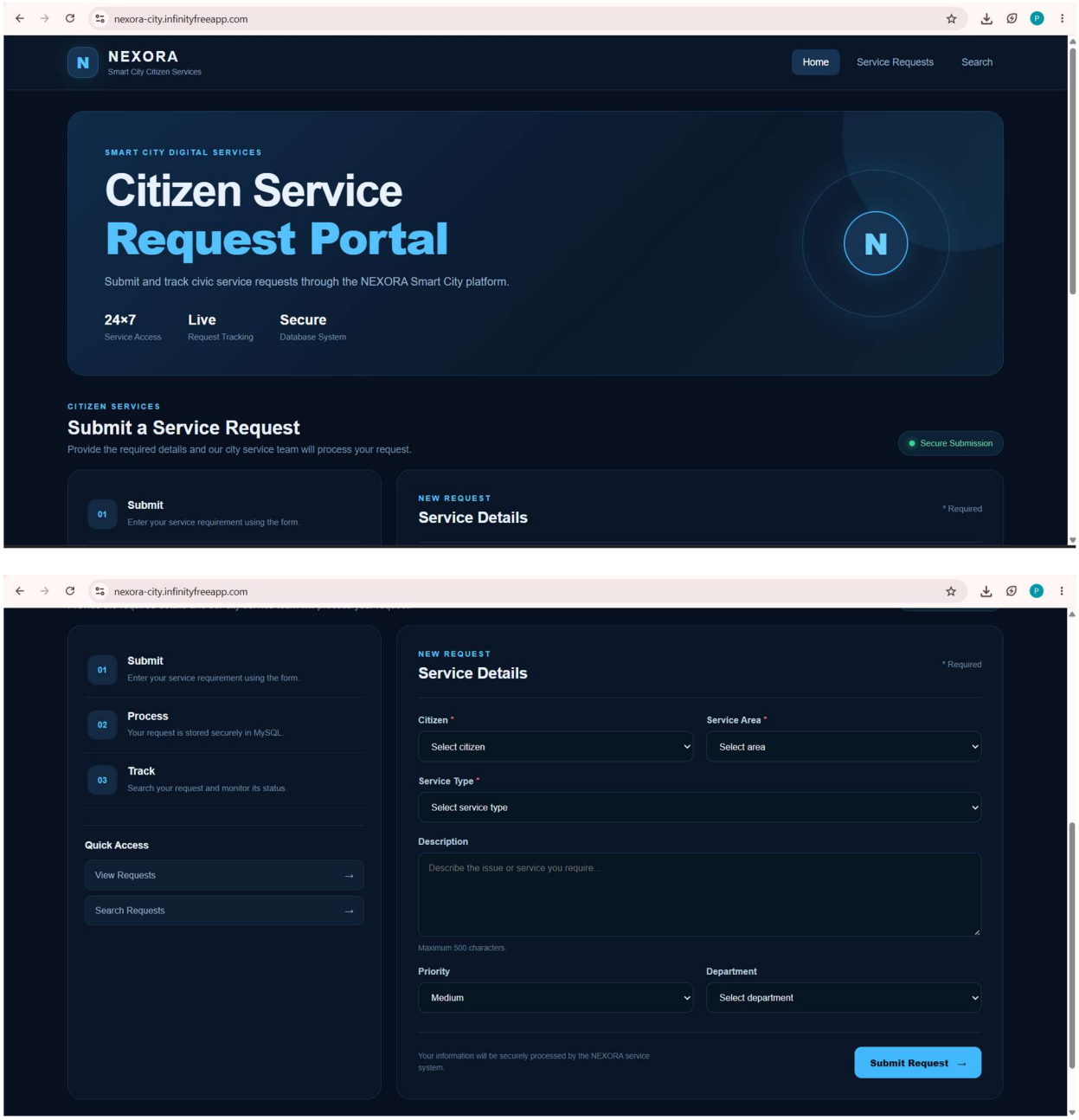


Hosted InfinityFree search/filter page using O'Reilly.



Screenshot 7 — Live InfinityFree URL in Browser

Browser address bar visibly shows the deployed nexora-city.infinityfreeapp.com domain.



16. Advantages of the System

- Easy and meaningful request submission for citizens.
- Structured data storage using a relational database.
- Faster retrieval and searching of request records.
- Online access through the hosted website.
- Prepared statements improve database security.
- Server-side validation improves reliability.
- Supports important DBMS operations required for the mini project.

17. Future Scope

- Citizen login and profile management.
- Request status tracking dashboard.
- Email/SMS notifications.
- Photo upload for issue evidence.
- Admin dashboard with update/delete operations.
- Department-wise analytics and reports.
- Interactive map-based issue reporting.

18. Conclusion

NEXORA successfully demonstrates how a secure database-driven website can be developed using HTML, CSS, PHP and MySQL. The system allows citizens to submit service requests, stores records in a relational database, displays saved data and provides search/filter functionality.

The project demonstrates important DBMS operations including INSERT and SELECT, while prepared statements, server-side validation and htmlspecialchars() provide safer handling of user input and output. The application has also been deployed on InfinityFree and tested successfully through live submission and security testing.

19. Declaration of Original Work

I hereby declare that the project titled "NEXORA – Smart City Citizen Service Request Management System" has been developed by me as part of my academic DBMS Unit 4 mini-project. The project has been created for educational purposes using HTML, CSS, PHP and MySQL and hosted using InfinityFree.

I declare that the implementation, database design, testing and documentation presented in this report represent my project work. Any external tools or services used during development have been used appropriately.